

FL-AQM Congestion Control algorithm:

1. Start

2. Define helper **membership functions**:

- Function **low**(x, a, b):
 - If $x \leq a$: return 1
 - If $x \geq b$: return 0
- Else: return linear ramp from 1 down to 0 between a and b
- Function **medium**(x, a, b, c):
 - If $x \leq a$ or $x \geq c$: return 0
 - If $x == b$: return 1
- Else: return linear ramp up from a→b and ramp down from b→c (triangle)
- Function **high**(x, a, b):
 - If $x \leq a$: return 0
 - If $x \geq b$: return 1
- Else: return linear ramp from 0 up to 1 between a and b

3. Function **fuzzy_congestion**(avg_queue, plr, arrival_rate, Q_max=100, AR_max=50):

- Normalize inputs to range $\sim[0,1]$:
 - $q = \text{avg_queue} / Q_max$
 - $p = \text{plr}$ (already 0..1)
 - $a = \text{arrival_rate} / AR_max$
- Fuzzify each normalized input using membership functions:
 - For queue q compute q_low , q_med , q_high with chosen breakpoints
 - For packet loss p compute p_low , p_med , p_high
 - For arrival rate a compute a_low , a_med , a_high
- Evaluate fuzzy rules (use min for AND, max for OR):
 - **rule_low** = $\min(q_low, p_low, a_low)$ all inputs low → low congestion
 - **rule_med** = $\max(\begin{array}{l} \blacksquare \min(q_med, p_low), \\ \blacksquare \min(q_low, p_med), \\ \blacksquare \min(q_med, p_med) \end{array})$
 -) several medium-ish combinations → medium
 - **rule_high** = $\max(\begin{array}{l} \blacksquare \min(q_high, p_high), \\ \blacksquare \min(q_high, a_high), \\ \blacksquare \min(p_high, a_high) \end{array})$
 -) any strong-high combination → high
- Defuzzify to a single congestion number (weighted average):
 - Assign representative numbers: low→0.2, med→0.5, high→0.9
 - **congestion** = $(\text{rule_low} \cdot 0.2 + \text{rule_med} \cdot 0.5 + \text{rule_high} \cdot 0.9) / (\text{rule_low} + \text{rule_med} + \text{rule_high} + \text{tiny})$
 - Return **congestion**

4. Function **active_queue_management**(congestion_level, drop_min=0.1, drop_max=0.6):

- If **congestion_level** < 0.3: **drop_prob** = 0.0
- Else if **congestion_level** < 0.6: **drop_prob** = drop_min + congestion_level * 0.3
- Else: **drop_prob** = drop_max
- Return **min(drop_prob, 1.0)**

5. Example run:

- Inputs: **avg_queue** = 65, **plr** = 0.18, **arrival_rate** = 40
- Call **congestion** = **fuzzy_congestion**(avg_queue, plr, arrival_rate)
- Call **drop_probability** = **active_queue_management**(congestion)
- **Print** congestion and drop_probability